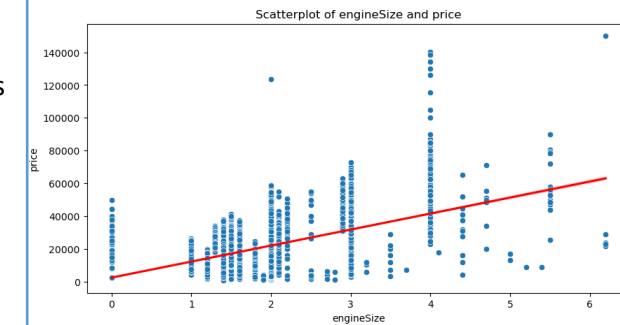
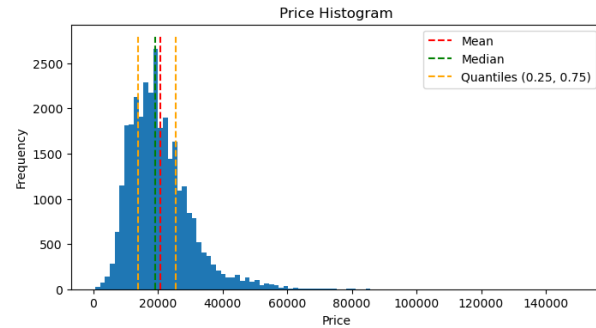
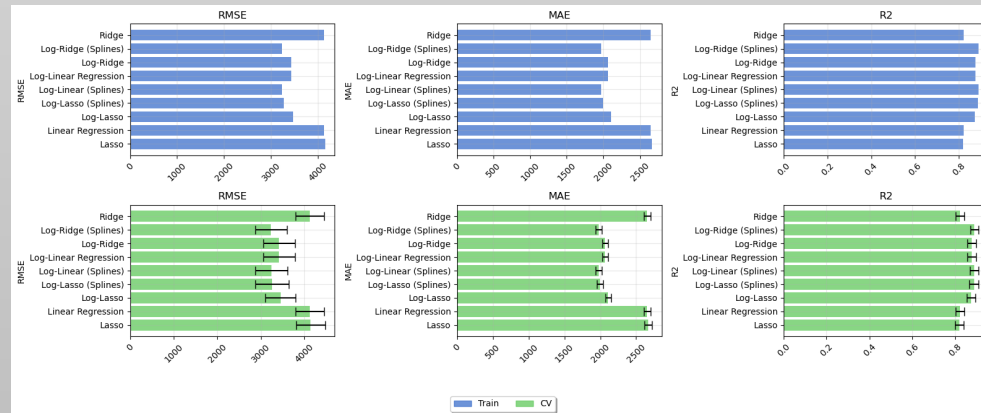


1. Lineare Modelle, später mit Splines¹ erweitert

- Analyse der Features und des Labels -> auffällig (leicht rechtsschiefe Verteilung des Preises, wodurch eine Log-Transformation möglicherweise besser performt -> vgl. Plot links)
- Lediglich OneHotEncoding der kategorialen Features und Standardisieren der numerischen (categorical_features = ['model', 'transmission', 'fuelType', 'brand']). 'engineSize' habe ich als numerisches Feature verwendet, da leicht steigend mit Preis (vgl. Plot rechts)
- Funktion für den Scorer erstellt (da teilweise log-transformierte Zielvariablen), Funktion für die Resultate (jeweils für das Training aber auch unter Berücksichtigung von CV KFold=5), Funktion für die Visualisierung der Resultate
- Lasso hat 8 Features auf 0 gesetzt, die ich entfernt habe für die linearen Modelle (14 bei Polynomen)
- Modelle mit Splines performten besser (gab Hinweis darauf, dass es nicht-linearen Zusammenhang zwischen Label sowie den Features gibt).
- Optimaler Grad der Polynome für jedes numerische Feature ermittelt

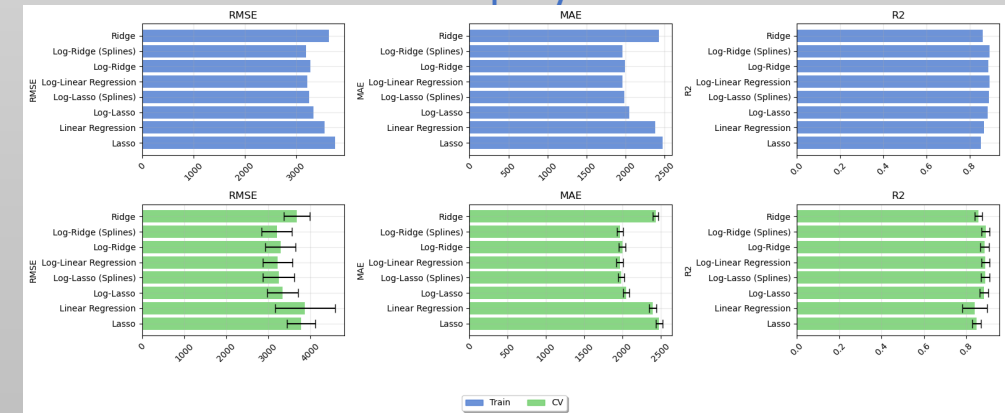


Lineare Modelle



	Model	RMSE	RMSE (SD)	MAE	MAE (SD)	R2	R2 (SD)
0	Log-Ridge (Splines)	3236.646281	359.206417	1977.167109	40.438322	0.890135	0.019456
0	Log-Linear (Splines)	3245.558876	364.219772	1979.419052	40.695273	0.889509	0.019854
0	Log-Lasso (Splines)	3261.683567	385.833686	1995.855410	41.676216	0.888310	0.021461
0	Log-Ridge	3423.263066	356.631792	2068.082822	41.441823	0.877180	0.020118
0	Log-Linear Regression	3423.336057	356.664066	2068.087352	41.450274	0.877175	0.020121
0	Log-Lasso	3458.038536	346.410049	2110.456239	40.758539	0.874723	0.019515
0	Ridge	4124.934828	328.219748	2655.720869	50.277428	0.822051	0.020429
0	Linear Regression	4124.952527	328.351100	2655.734613	50.382340	0.822049	0.020441
0	Lasso	4149.001958	327.986370	2669.221528	50.551774	0.819971	0.020507

Lineare Modelle mit polynomialen Features



	Model	RMSE	RMSE (SD)	MAE	MAE (SD)	R2	R2 (SD)
0	Log-Ridge (Splines)	3200.740276	358.733536	1965.695035	41.715060	0.892535	0.019293
0	Log-Linear Regression	3218.656695	355.045013	1967.271811	45.028924	0.891340	0.019186
0	Log-Lasso (Splines)	3244.420679	372.907390	1986.184553	41.579536	0.889536	0.020482
0	Log-Ridge	3289.114411	363.993397	1998.610882	43.140941	0.886536	0.020070
0	Log-Lasso	3336.702288	370.157406	2052.025428	40.117556	0.883218	0.020707
0	Ridge	3676.269411	311.873973	2432.881682	39.848038	0.858595	0.017766
0	Lasso	3779.686431	342.268058	2477.514003	44.091747	0.850462	0.020449
0	Linear Regression	3873.265810	708.466469	2395.189355	46.333915	0.839380	0.058244

Fazit:

- Polynomiale Features performten nur leicht besser als die Modelle mit Splines, da beide Methoden ähnliches erzeugen (Hauptunterschied: polynomiale Features global, während Splines lokal)
- Log-Transformation verbesserte Leistung. Zeigt, dass Label vermutlich nichtlineare Verteilung hat, die durch log-Transformation besser modelliert werden kann
- Ich muss Modelle verwenden, die mit nicht-linearen Zusammenhänge umgehen können

¹Splines bieten oft grössere Flexibilität und bessere numerische Stabilität, während polynomiale Features einfacher zu interpretieren sind. Hauptunterschied: polynomiale Features sind globale Transformationen, die im gesamten Wertebereich eines Features gleich wirken, während Splines lokal arbeiten.

2. RandomForestRegressor und XGBRegressor



- Mit dem gleichen Dataframe wie zuvor obig genannte Modelle trainiert. Vor allem beim Hyperparameter Tuning habe ich viel Zeit investiert und mich über die verschiedenen Algorithmen informiert. Ich habe GridSearchCV, RandomizedSearchCV sowie BayesSearchCV verwendet. Auch mit Optuna habe ich experimentiert, jedoch erfolglos. Zum Teil habe ich die Hyperparametersuche auf Google Colab laufen lassen, da GPU. Insgesamt mehrere Tage Zeit investiert, was Geduld und Nerven gekostet hat.
- Systematischer Vorgang bei Hyperparametersuche: Mit RandomizedSearchCV (mit 100 Kombinationen) zunächst «grobe» Hyperparameter definiert, wobei ich diesen Raum dann immer «granularer» gestaltete mit GridSearchCV.
- In Recherche zufälligerweise über den BayesSearchCV-Algorithmus gelesen, der basierend auf den bereits im Training verwendeten Hyperparameter lernt, welche Kombinationen noch «sinnvoll» wären. Diese Hyperparameterkombinationen vom RandomizedSearchCV habe ich sodann mit BayesSearchCV granularer definiert (engeres Intervall um diese Parameter).
- Mit dem BayesSearchCV und dem XGBRegressor habe ich den geringsten RMSE bei der CV erzielt. Schliesslich habe ich damit auch den besten Score auf Kaggle erzielt (Name: **Test1920**)

Vor-/Nachteile RandomForestRegressor:

- + leicht zu verstehen und implementieren
- + bietet Infos zur Wichtigkeit der Features, was zur Interpretierbarkeit beiträgt
- + im Vergleich zu XGBoost weniger Hyperparameter-Tuning erforderlich
- Bei hochkomplexen Datensätzen können schlechtere Ergebnisse erzielt werden, da er weniger gezielt optimiert
- Lernt im Gegensatz zu Boosting nicht iterativ, wodurch Schwächen nicht gezielt korrigiert werden

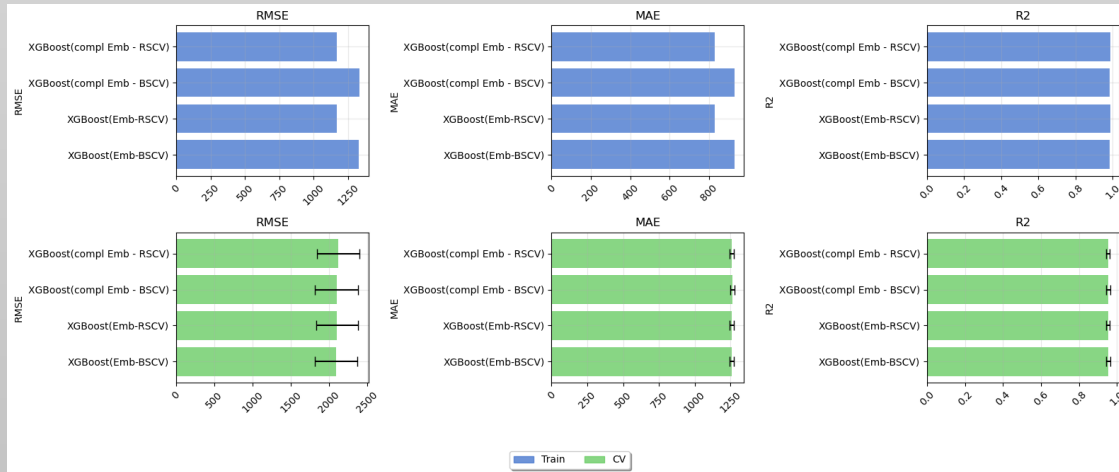
Vor-/Nachteile XGBRegressor:

- + Effizient durch Gradient Boosting, wodurch Modelle iterativ verbessert werden
- + viele verschiedene Hyperparameter, die eine präzise Anpassung ermöglichen
- + Regularisierungsparameter 'alpha' und 'lambda'
- Wegen komplexeren Berechnungen rechenintensiver
- Sorgfältiges Feintuning der Hyperparameter, was zeitaufwändig und technisch anspruchsvoll ist
- Ist komplexer und schwerer zu interpretieren als RF

Fazit:

- Die nicht-linearen Modelle erzielten massiv bessere Ergebnisse als die linearen Modelle. Das beste Modell, XGBoost mit BayesSearchCV Hyperparameter-Tuning erzielte ein RMSE bei der CV von 2'097, wobei die Standardabweichung akzeptabel war. Zu beobachten war auch ein starkes Overfitting, was bei Modellen wie XGBoost oder RF häufig passieren kann. Da ich mein Modell jedoch basierend auf dem CV RMSE aussuche, ist das Overfitting «nicht allzu schlimm».
- Ich wollte jedoch auch noch gewisse Features mit Text-Encoding bearbeiten, um zu sehen, ob ich damit ein besseres Ergebnis erzielen kann.

3. Feature Processing mit Textmining



	Model	RMSE	RMSE (SD)	MAE	MAE (SD)	R2	R2 (SD)
0	XGBoost(Embeddings-BayesSearchCV)	2098.127627	277.841448	1259.736624	14.057816	0.953655	0.010329
0	XGBoost(Complex Embeddings - BayesSearchCV)	2103.587882	281.856703	1263.979784	15.828304	0.953395	0.010575
0	XGBoost(Embeddings-RandomizedSearchCV)	2109.116708	271.228555	1257.447098	14.462228	0.953188	0.010104
0	XGBoost(Complex Embeddings - RandomizedSearchCV)	2121.089030	276.554497	1260.045046	15.809991	0.952640	0.010421

- Wir sehen, dass wir durch das Feintuning der Hyperparameter mit BayesSearchCV den RMSE auf 2'098 senken konnten. Damit ist das Ergebnis ähnlich wie ohne Embeddings für das Feature «model». Nun ist die Standardabweichung bei der CV leicht geringer bei 277 statt 285. Wenn wir unsere Entscheidung rein auf dem RMSE (ohne SD) treffen, müssen wir somit das XGBRegressor-Modell ohne Textembeddings wählen (vgl. rot umrahmtes Modell Folie 2). Aus wirtschaftlicher Sicht würde dies auch Sinn ergeben, da das Bilden von Text-Embeddings sehr rechen- und zeitintensiv ist, was mit höheren Kosten verbunden ist.
- Auf Kaggle habe ich natürlich die Predictions beider Herangehensweisen eingereicht, wobei mit der Variante ohne Embeddings ein RMSE von 1'888 und mit Embeddings ein RMSE von 1'893 erzielt wurde. Dies bestätigt auch obigen Punkt, dass die beiden Modelle praktisch gleich auf ungesehenen Daten performen.

- Anwendung von Embeddings auf die Feature model und brand mit dem Modell «all-MiniM-L6-v2» sowie «microsoft/mpnet-base». Encoding von brand stellt sich jedoch nicht als sinnvoll heraus, da es nur wenige Klassen gab. Mit erstem Modell hatte ich somit total (ohne OneHotEncoding) 393 Spalten (vgl. auf Abbildung «Emb»), mit letzterem über 700 (vgl. auf Abbildung «comp Emb»). Ausserdem habe ich beim SentenceTransformer «normalize_embeddings = True» gesetzt, damit die Embeddings bereits normalisiert sind. Auf meinem MacBook Pro mit Intel Chip i5 stürzte mein Kernel die ganze Zeit ab. Deswegen habe ich das Notebook über GoogleColab weiterlaufen lassen, da hier GPU z.V. stehen.
- Auch das Hyperparametertuning mit BayesSearchCV (mit über 100 Iterationen) führte zum «Sterben» des Kernels (ein Mal nach 50min Training, was sehr frustrierend war). Deswegen habe ich auch hier GoogleColab verwendet. Die Vorgehensweise fürs Hyperparameter-Tuning war wieder wie zuvor. Jedoch brauchte es dieses Mal deutlich mehr Zeit, da das Dataset grösser war (z.T. pro BayesSearchCV über 3.5 Stunden über die Nacht).
- Deswegen habe ich ebenfalls meine Modelle ausprobiert, in dem ich die Dimension der Embeddings mittels «umap» reduziert habe (auf 2, 3, 5, 10, 20, 50 sowie 100). Auch dies hat extrem viel Zeit gekostet, da ich danach mit der reduzierten Dimension jeweils wieder die optimalen Hyperparameter finden musste. Mit den reduzierten Dimensionen erzielte ich leider keine guten Ergebnisse.
- Deswegen sind hier, nach dem Ausprobieren so viel möglicher unterschiedliche Herangehensweisen an die Modelle, lediglich die Ergebnisse der XGBRegressor-Modelle, bei denen ich einmal mittels «all-MiniM-L6-v2» und einmal mittels «microsoft/mpnet-base» Embeddings gebildet habe, dargestellt. Jeweils mit Hyperparameter-Tuning über RandomizedSearchCV sowie BayesSearchCV.

Fazit allgemein:

- Viel gelernt, besonders über das Hyperparameter-Tuning, das Text-Encoding und Computerleistung (Arbeitsspeicher etc.)
- Jedoch hat dies extrem viel Geduld und Zeit gekostet (mehrere Tage und Nächte)
- Um die Vorhersagequalität weiter zu steigern, würde ich mich mehr in das Hyperparameter-Tuning mit Optuna einlesen. Vielleicht würde das Kombinieren gewisser Features (z.B. Brand und Model) ein besseres Ergebnis liefern. Ausserdem könnten weitere Gradient Boosting Modelle, wie LightGBM verwendet werden. Schliesslich würde das Trainieren eines Neuronalen Netzwerkes bestimmt auch Sinn ergeben, wenn man ein möglichst gutes Resultat erzielen möchte. Dabei sollte jedoch auch die Zeit, die Kosten und die zusätzliche Komplexitätssteigerung berücksichtigt werden.